

Implementation Instructions

Reinforcement Learning Solution Methodology for CIPP and RCIPP

Prepared by: Dr. Mesut Sayin

June 12, 2026

1 Purpose of This Document

This document provides detailed step-by-step instructions for implementing a reinforcement learning (RL) based solution methodology for the Campaign Itinerary Planning Problem (CIPP) and its robust counterpart (RCIPP).

The goal of this project is not to treat RL as a minor extension. Instead, this project should position CIPP/RCIPP as the main optimization models and RL as a data-driven solution methodology for solving them, especially under long horizons, large state sets, and dynamic campaign environments.

The Research Assistant is expected to implement the RL framework from start to finish within two months.

2 High-Level Research Goal

The project aims to answer the following research question:

Can a reinforcement learning agent learn high-quality campaign itinerary policies for CIPP and RCIPP, while respecting all feasibility constraints and adapting efficiently to dynamic reward, budget, and shock information?

The RL methodology should be compared against:

- (i) deterministic CIPP solved by Gurobi,
- (ii) robust CIPP solved by Gurobi,
- (iii) deterministic A-VLNS,
- (iv) robust A-VLNS,
- (v) deterministic reoptimization,
- (vi) robust reoptimization.

The final output should be a reproducible Python codebase and a set of computational results suitable for an academic report.

3 Core Modeling Background

3.1 Deterministic CIPP

The deterministic CIPP chooses where and when a candidate should campaign. Let:

- $I = \{1, \dots, n\}$ be the set of campaign locations, e.g., states.
- $T = \{1, \dots, t_{\max}\}$ be the campaign horizon.
- $Z_{it} = 1$ if a visit is made to location i on day t .
- $V_t = 1$ if day t is idle.

The deterministic objective is approximately:

$$\max \sum_{i \in I} \sum_{t \in T} \lambda_t \mu_{n_i} \pi_i Z_{it},$$

where:

- π_i is the base reward of visiting state i ,
- λ_t is the time-dependent reward multiplier,
- μ_{n_i} is the repeat-visit penalty,
- $n_i = \sum_{t \in T} Z_{it}$ is the total number of visits to state i .

The major constraints are:

1. At most one state visit per day when $p = 1$.
2. A minimum number of idle days in every rolling α -day window.
3. At most q total visits to each state.
4. At most w visits to the same state in every rolling α -day window.
5. Total campaign cost cannot exceed budget b .

3.2 Robust CIPP

The robust model assumes rewards and budget are uncertain. The nominal reward of state i is $\bar{\pi}_i$, and the maximum downward deviation is $\hat{\pi}_i$. The robust reward is evaluated as:

$$\sum_{i \in I} \sum_{t \in T} \lambda_t \mu_{n_i} \bar{\pi}_i Z_{it} - P_{\pi}(Z),$$

where $P_{\pi}(Z)$ is the worst-case reward loss under a Γ -budgeted uncertainty set.

The robust budget constraint is:

$$\sum_{i \in I} \sum_{t \in T} c_i Z_{it} \leq \bar{b} - \Gamma_b \hat{b}.$$

For the RA: think of the robust model as a more conservative version of the deterministic model. It protects the itinerary against unfavorable reward and budget realizations.

4 RL Formulation

4.1 RL Environment

Each RL episode represents one complete campaign. The episode starts at day $t = 1$ and ends at $t = t_{\max}$.

At each day t , the RL agent observes the current campaign state, selects an action, receives a reward, and the environment moves to the next day.

4.2 State Representation

The state vector s_t must include all information needed to determine both reward and feasibility.

At minimum, include:

$$s_t = [t, B_t, n_1(t), \dots, n_n(t), r_1(t), \dots, r_n(t), h_t, \lambda_t, \pi_{1t}, \dots, \pi_{nt}],$$

where:

- t is the current day.
- B_t is remaining budget.
- $n_i(t)$ is the number of previous visits to state i .
- $r_i(t)$ is the number of visits to state i in the current rolling α -day window.
- h_t is the number of idle days in the current rolling α -day window.
- λ_t is the time-dependent reward multiplier.
- π_{it} is the current reward estimate for state i .

For robust RL, also include:

$$\hat{\pi}_{1t}, \dots, \hat{\pi}_{nt}, \quad \Gamma_{\pi,t}, \quad \Gamma_{b,t}, \quad B_t^R.$$

4.3 Action Space

For the base implementation, assume $p = 1$. Then the action space is:

$$A = I \cup \{\text{Idle}\}.$$

If there are n states, the action space has $n + 1$ actions:

$$0 = \text{Idle}, \quad 1, \dots, n = \text{visit state } i.$$

4.4 Feasible Action Masking

This is one of the most important implementation requirements.

The RL agent must not be allowed to choose infeasible actions. At each day t , the environment must construct a binary mask:

$$m_t(a) = \begin{cases} 1, & \text{if action } a \text{ is feasible,} \\ 0, & \text{otherwise.} \end{cases}$$

An action “visit state i ” is infeasible if any of the following holds:

1. The total number of visits to i has already reached q :

$$n_i(t) \geq q.$$

2. Visiting i would violate the rolling-window visit cap:

$$r_i(t) \geq w.$$

3. The remaining budget is insufficient:

$$c_i > B_t.$$

4. Visiting today would make it impossible to satisfy future idle-day requirements inside the active rolling window.

The idle action is infeasible if choosing idle would cause a problem elsewhere. Usually idle is feasible unless it conflicts with a modeling rule. In the base implementation, keep idle feasible whenever possible.

The RA must implement action masking before training the RL algorithm. This prevents the RL agent from learning illegal campaign schedules.

4.5 Deterministic RL Reward

If the agent visits state i on day t , the immediate reward should be:

$$r_t = \lambda_t \mu_{n_i(t)+1} \pi_i.$$

If the action is idle, then:

$$r_t = 0.$$

The repeat-visit factor is:

$$\mu_j = 1 - \gamma j.$$

If desired, to make the agent more cost-aware, use:

$$r_t = \lambda_t \mu_{n_i(t)+1} \pi_i - \beta_c c_i,$$

but the first implementation should keep cost only as a hard feasibility constraint.

4.6 Robust RL Reward

For robust RL, use the nominal reward minus an uncertainty exposure penalty:

$$r_t^R = \lambda_t \mu_{n_i(t)+1} \bar{\pi}_i - \omega \lambda_t \mu_{n_i(t)+1} \hat{\pi}_i.$$

Here, ω is a tunable penalty coefficient. Start with:

$$\omega \in \{0.25, 0.50, 1.00\}.$$

In addition, at the end of the episode, compute the exact RCIPP-style robust objective by calculating the worst-case reward loss $P_\pi(Z)$. This final robust objective must be used for reporting and comparison.

5 How CIPP and RCIPP Are Used

5.1 Role 1: Feasibility Definition

The deterministic CIPP constraints define the environment feasibility rules. The RL agent must operate inside the CIPP feasible region through action masking.

5.2 Role 2: Benchmarking

The RL method must be compared with deterministic CIPP and RCIPP solutions. For small instances, Gurobi can provide optimal or near-optimal solutions. For large instances, compare RL with A-VLNS and robust A-VLNS.

5.3 Role 3: Imitation Learning Teacher

For small instances where Gurobi solves CIPP optimally, extract state-action pairs from the optimal itinerary:

$$(s_t, a_t^*), \quad t = 1, \dots, t_{\max}.$$

Use these pairs to pre-train the RL policy by supervised learning. This gives the RL agent a strong starting point.

5.4 Role 4: Warm Start and Repair

The RL-generated itinerary can be used as a warm start for A-VLNS or Gurobi. If the RL itinerary is feasible but not locally optimized, use A-VLNS to improve it.

The hybrid method is:

$$\text{RL constructs itinerary} \rightarrow \text{A-VLNS improves itinerary.}$$

5.5 Role 5: Robust Evaluation

Even when the RL policy is trained using deterministic rewards, the final itinerary should be evaluated under both:

- deterministic objective value,
- robust objective value.

This allows us to test whether RL produces high-reward but fragile schedules, or whether it naturally diversifies visits.

6 RL-Guided Reoptimization

In addition to direct sequential itinerary construction, the project should implement an RL-guided reoptimization module.

6.1 Baseline Reoptimization Logic

In the existing CIPP/RCIPP framework, reoptimization occurs when a material event or update is observed. For example:

- reward of a state changes,
- budget changes,
- a planned event becomes infeasible,
- a shock affects battleground states.

The reoptimization model fixes all past decisions and optimizes only the remaining horizon.

6.2 RL Meta-Agent

The RL-guided reoptimization layer should learn whether and how to reoptimize.

At each day t , the meta-agent chooses:

$a_t^{meta} \in \{\text{No Reoptimization, Deterministic Reoptimization, Robust Reoptimization, A-VLNS Repair}\}.$

The meta-agent state should include:

- current day t ,
- remaining horizon,
- size of reward changes,
- number of affected states,
- current budget pressure,
- current robust uncertainty level,
- number of future planned visits that would change,
- time since last reoptimization,
- estimated value loss if no reoptimization is performed.

The meta-agent reward should be:

$$r_t^{meta} = \Delta \text{Objective} - \psi \times \text{Number of revised future visits} - \kappa \times \text{Computational cost}.$$

6.3 Simplified First Implementation

Do not begin with a complex meta-agent. First implement a rule-based reoptimization trigger:

$$\chi_t = \begin{cases} 1, & \text{if reward change exceeds threshold,} \\ 1, & \text{if budget change exceeds threshold,} \\ 1, & \text{if planned action becomes infeasible,} \\ 0, & \text{otherwise.} \end{cases}$$

Then replace this rule with an RL meta-agent in Week 7.

7 Required Code Architecture

The RA should organize the codebase as follows:

```
cipp_rl_project/  
  
data/  
  raw/  
  processed/  
  instances/  
  
src/
```

```
envs/
  cipp_env.py
  rcipp_env.py
  reopt_env.py

models/
  dqn_agent.py
  ppo_agent.py
  imitation_model.py
  meta_reopt_agent.py

optimization/
  cipp_gurobi.py
  rcipp_gurobi.py
  avlms.py
  repair.py

utils/
  instance_generator.py
  evaluation.py
  plotting.py
  masks.py

experiments/
  train_dqn.py
  train_ppo.py
  train_imitation.py
  run_reopt.py
  compare_methods.py

results/
  logs/
  models/
  tables/
  figures/

notebooks/

requirements.txt
README.md
main.py
```

8 Algorithms to Implement

8.1 Algorithm 1: Feasibility-Masked RL for CIPP

1. Initialize campaign environment.
2. For each episode:
 - (a) Set $t = 1$.
 - (b) Set all visit counts to zero.

- (c) Set remaining budget to b .
- (d) While $t \leq t_{\max}$:
 - i. Construct state s_t .
 - ii. Construct feasible action mask m_t .
 - iii. RL agent selects feasible action a_t .
 - iv. Environment updates visit counts, budget, rolling windows, and idle history.
 - v. Environment returns reward r_t .
 - vi. Store transition $(s_t, a_t, r_t, s_{t+1}, m_t)$.
- (e) Compute final deterministic and robust objective values.
- (f) Update RL policy.

8.2 Algorithm 2: RL with Imitation Learning Warm Start

1. Solve small CIPP instances with Gurobi.
2. Extract optimal state-action pairs.
3. Train neural network policy by supervised learning.
4. Use trained policy parameters to initialize the RL agent.
5. Continue RL training using environment interactions.

8.3 Algorithm 3: RL-Guided Reoptimization

1. Generate initial itinerary using RL, CIPP, or RCIPP.
2. For each day t :
 - (a) Execute current action.
 - (b) Observe updates or shocks.
 - (c) Construct meta-state.
 - (d) Meta-agent chooses:

$$\{\text{No Reopt, CIPP-ReOpt, RCIPP-ReOpt, A-VLNS Repair}\}.$$
 - (e) If reoptimization is selected:
 - i. Fix past decisions.
 - ii. Update rewards and budget.
 - iii. Reoptimize remaining horizon.
 - iv. Penalize schedule revisions.

9 RL Methods to Implement

9.1 Phase 1: DQN

Start with Deep Q-Network because the action space is discrete.

The Q-function is:

$$Q_{\theta}(s, a).$$

The policy chooses:

$$a_t = \arg \max_{a \in A_t^{feas}} Q_\theta(s_t, a).$$

Important: infeasible actions must receive value $-\infty$ before taking the maximum.

9.2 Phase 2: PPO

After DQN works, implement PPO. PPO is generally more stable and may work better for larger instances.

The policy is:

$$\pi_\theta(a|s).$$

The action mask must be applied before sampling from the policy distribution.

9.3 Phase 3: Imitation + RL

Use Gurobi-generated optimal solutions to pre-train the policy. Then continue training with PPO or DQN.

10 Experiments

10.1 Instance Classes

Use the same instance structure as the CIPP paper:

- Number of states: $n \in \{14, 30, 51\}$.
- Number of periods: $t_{\max} \in \{30, 45, 60, 75, 90\}$.
- Party type: Democrat and Republican instances.

10.2 Methods to Compare

1. Gurobi-CIPP.
2. Gurobi-RCIPP.
3. A-VLNS-CIPP.
4. A-VLNS-RCIPP.
5. RL-DQN.
6. RL-PPO.
7. Imitation + PPO.
8. RL + A-VLNS repair.
9. RL-guided reoptimization.
10. RL-guided robust reoptimization.

10.3 Performance Metrics

Report the following metrics:

- Deterministic objective value.
- Robust objective value.
- Optimality gap versus Gurobi where available.
- Improvement over A-VLNS where available.
- Runtime.
- Budget utilization.
- Number of idle days.
- Number of unique states visited.
- Herfindahl-Hirschman Index for geographic concentration.
- Number of schedule revisions in reoptimization.
- Reward exposure under uncertainty.
- Constraint violations. This should always be zero.

11 Two-Month Timeline

11.1 Weekly Milestones

Week	Main Goal	Deliverables
1	Understand CIP-P/RCIPP and build instance generator	Read model description carefully. Implement data structures for states, periods, rewards, costs, budgets, rolling windows, and repeat-visit counters. Produce synthetic instances and reproduce deterministic objective calculation.
2	Implement CIPP RL environment	Create <code>CIPPEnv</code> . Implement reset, step, state construction, reward calculation, feasibility checks, and action masks. Test manually on small instances. Constraint violations must be zero.
3	Implement DQN baseline	Train a DQN agent on small instances. Include action masking. Produce learning curves. Compare against random feasible policy and greedy policy.
4	Implement PPO and imitation learning	Implement PPO with action masking. Solve small instances with Gurobi and extract state-action pairs. Pre-train policy by imitation learning. Compare DQN, PPO, and imitation+PPO.
5	Implement robust RL environment	Create <code>RCIPPEnv</code> . Add nominal rewards, reward deviations, robust budget, and robust objective evaluation. Train robust RL policies under moderate and conservative uncertainty.

Week	Main Goal	Deliverables
6	Benchmarking and hybrid repair	Compare RL methods with CIPP, RCIPP, and A-VLNS. Implement RL warm start for A-VLNS. Produce tables for all instance sizes.
7	RL-guided reoptimization	Implement dynamic shocks and reoptimization environment. First implement rule-based trigger, then meta-RL trigger. Compare no reoptimization, rule-based reoptimization, and RL-guided reoptimization.
8	Final experiments and documentation	Run final experiments. Produce clean tables, figures, logs, trained models, README, and implementation report. Prepare results for final report.

11.2 Gantt Chart

12 Detailed Weekly Instructions

12.1 Week 1: Optimization Understanding and Data Structures

Tasks

1. Read the CIPP and RCIPP model descriptions.
2. Implement a Python class called `CIPPInstance`.
3. Store the following:
 - number of states n ,
 - number of periods $t_{\max}$,
 - reward vector π ,
 - cost vector c ,
 - budget b ,
 - rolling window length α ,
 - minimum idle requirement $u(t)$,
 - total visit limit q ,
 - rolling visit cap w ,
 - temporal weights λ_t ,
 - repeat penalty parameter γ .
4. Implement deterministic objective evaluator.
5. Implement feasibility checker.

Definition of Done

By the end of Week 1, the RA must be able to input a complete itinerary and return:

- objective value,
- total cost,
- number of idle days,

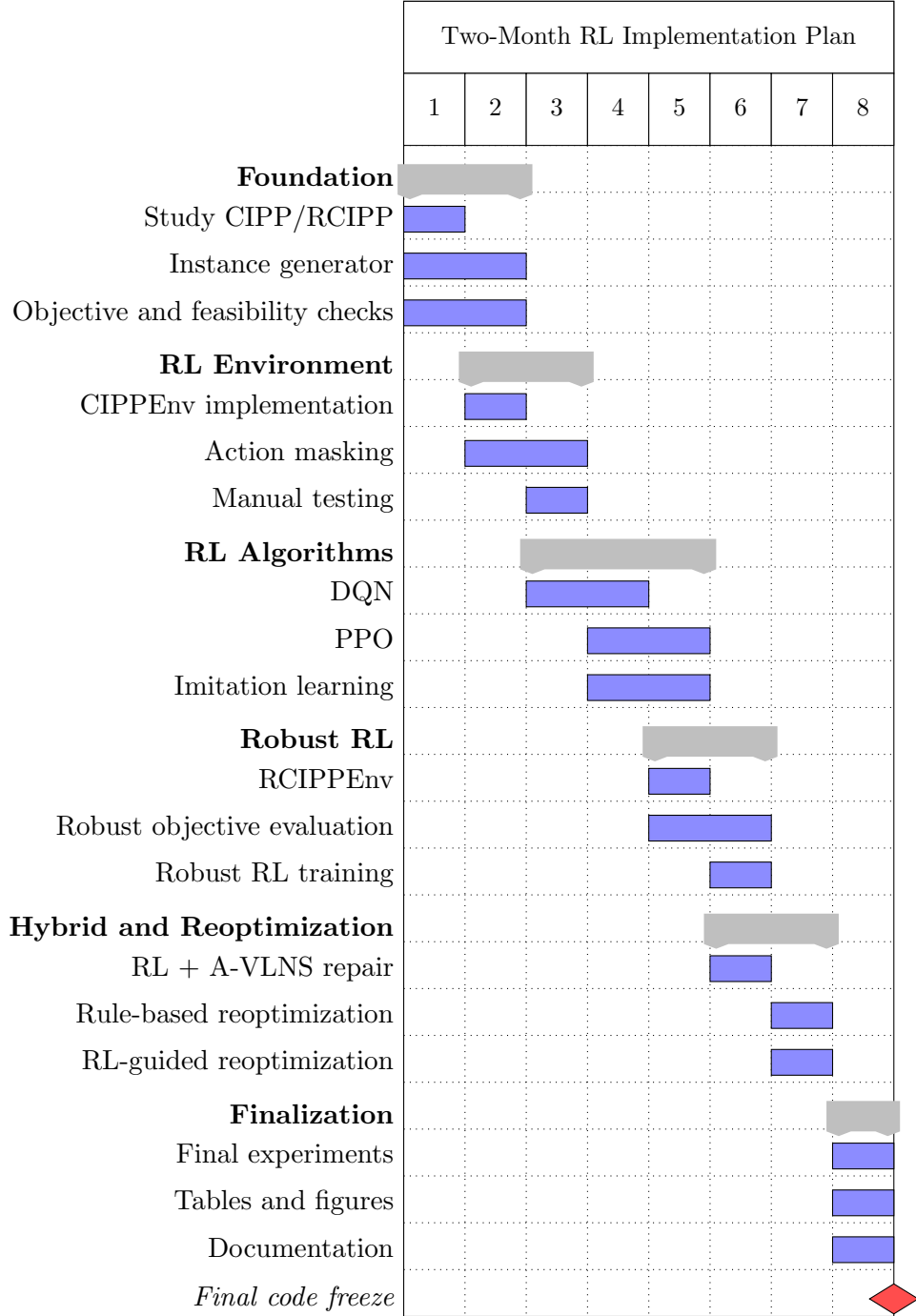


Figure 1: Implementation schedule.

- visit count per state,
- whether the itinerary is feasible,
- list of violated constraints, if any.

12.2 Week 2: CIPP Environment

Tasks

Implement an OpenAI-Gym-style environment:

```
class CIPPEnv:
    def reset(self):
        ...
    def step(self, action):
        ...
    def get_action_mask(self):
        ...
    def compute_reward(self, action):
        ...
    def is_done(self):
        ...
```

The environment must:

- maintain current day,
- maintain visit history,
- maintain rolling-window history,
- maintain remaining budget,
- produce action masks,
- return next state and reward.

Definition of Done

The RA must generate 1,000 random feasible episodes using only the action mask. There must be zero constraint violations.

12.3 Week 3: DQN Baseline

Tasks

1. Implement DQN with replay buffer.
2. Apply action masking before action selection.
3. Train on small instances:

$$n = 14, \quad t_{\max} = 30.$$

4. Compare with:
 - random feasible policy,
 - greedy reward policy,
 - Gurobi solution if available.

Definition of Done

The DQN policy must outperform random feasible policy and should approach greedy policy. The RA must produce learning curve plots.

12.4 Week 4: PPO and Imitation Learning

Tasks

1. Implement PPO with categorical action distribution.
2. Apply action mask before sampling.
3. Solve small CIPP instances with Gurobi.
4. Convert optimal solutions into state-action pairs.
5. Train policy using cross-entropy loss:

$$\mathcal{L}(\theta) = - \sum_t \log \pi_\theta(a_t^* | s_t).$$

6. Use imitation-trained policy as PPO initialization.

Definition of Done

The RA must produce a comparison table:

DQN vs PPO vs Imitation+PPO vs Greedy vs Gurobi.

12.5 Week 5: Robust RL Environment

Tasks

1. Implement RCIPPEnv.
2. Add:
 - nominal rewards $\bar{\pi}_i$,
 - deviation values $\hat{\pi}_i$,
 - reward uncertainty budget Γ_π ,
 - budget uncertainty level Γ_b ,
 - robust budget.
3. Implement robust episode evaluation.
4. Train RL under:
 - nominal setting,
 - moderate uncertainty,
 - conservative uncertainty.

Definition of Done

The RA must report:

- nominal objective,
- robust objective,
- reward exposure,
- budget utilization,
- HHI concentration.

12.6 Week 6: Benchmarking and Hybrid Repair

Tasks

1. Run trained RL policies on all 30 benchmark instances.
2. Implement RL-generated warm starts for A-VLNS.
3. Compare:
 - pure RL,
 - RL + A-VLNS repair,
 - A-VLNS alone,
 - Gurobi where available.

Definition of Done

The RA must prepare tables with objective values, gaps, runtime, and feasibility status.

12.7 Week 7: RL-Guided Reoptimization

Tasks

1. Implement campaign shock generator.
2. Implement rule-based reoptimization trigger.
3. Implement meta-agent with actions:

{No Reopt, CIPP-ReOpt, RCIPP-ReOpt, A-VLNS Repair}.

4. Compare:
 - no reoptimization,
 - rule-based reoptimization,
 - RL-guided reoptimization.

Definition of Done

The RA must produce at least one shock experiment similar to Hurricane Helene-style analysis, showing how RL-guided reoptimization changes the future itinerary.

12.8 Week 8: Finalization

Tasks

1. Run final experiments.
2. Save all random seeds.
3. Generate final tables and figures.
4. Write README.
5. Clean code.
6. Prepare implementation report.

Definition of Done

The final project folder must contain:

- clean Python source code,
- reproducible experiments,
- trained models,
- result tables,
- learning curves,
- comparison plots,
- final implementation report.

13 Important Implementation Rules

1. Feasibility is non-negotiable. RL must never output infeasible final schedules.
2. Always use action masking.
3. Always report constraint violations.
4. Separate training objective from evaluation objective.
5. Always evaluate final solutions using the exact CIPP or RCIPP objective.
6. Use fixed random seeds.
7. Save every trained model.
8. Save every experiment configuration.
9. Start small before scaling.
10. Do not begin robust or reoptimization experiments before deterministic CIPP RL works correctly.

14 Expected Project Contribution

This project can be positioned as follows:

We develop a reinforcement learning based solution methodology for the Campaign Itinerary Planning Problem and its robust counterpart. The proposed method learns sequential campaign itinerary policies while preserving the structural feasibility requirements of the original integer programming models through action masking. We further develop imitation-enhanced RL and RL-guided reoptimization mechanisms to improve scalability and adaptability under dynamic campaign shocks.

The main methodological contributions are:

1. A feasibility-masked RL environment for CIPP.
2. A robust RL environment for RCIPP.
3. Imitation learning from exact CIPP solutions.
4. RL warm starts for A-VLNS.
5. RL-guided deterministic and robust reoptimization.
6. A computational comparison against Gurobi and A-VLNS.

15 Your Daily Reporting Template

At the end of each working day, you must send a short update using this format to my email: mesut.sayin@ozyegin.edu.tr.

```
Date:
Hours worked:

Tasks completed today:
1.
2.
3.

Files changed:
1.
2.
3.

Tests run:
1.
2.

Problems encountered:
1.
2.

Questions for PI:
1.
2.

Plan for tomorrow:
1.
2.
3.
```

16 Final Notes for the Research Assistant

The first objective is not to build the most complex RL model. The first objective is to build a correct and reliable environment. If the environment is wrong, all RL results will be meaningless.

The correct implementation order is:

Feasibility checker → CIPP environment → Action mask → DQN → PPO → Imitation learning
→ Robust RL → Hybrid repair → RL-guided reoptimization.

The most important deliverable is a fully reproducible experimental pipeline that can generate all tables and figures.